

Pratique de la Data Science

Cours 2 : Apprentissage supervisé.

Université Paris Dauphine – Master 2 Ingénierie Statistique et Financière (ISF)

Pierre Fihey – *pierre.fihey@ip-paris.fr*

Plan du cours

I. Introduction à l'apprentissage supervisé

- Problèmes de Classification
- Problèmes de Régression

II. Principaux modèles de Machine Learning

- Descente de Gradient
- Régressions linéaire et logistique
- Méthodes avancées

III. Métriques d'évaluation

IV. Explicabilité

Plan du cours

I. Introduction à l'apprentissage supervisé

- Problèmes de Classification
- Problèmes de Régression

II. Principaux modèles de Machine Learning

- Descente de Gradient
- Régressions linéaire et logistique
- Méthodes avancées

III. Métriques d'évaluation

IV. Explicabilité

Introduction à l'apprentissage supervisé

Setup : Chaque observation est une paire (x, y) avec $x \in \mathcal{R}^d$ les variables explicatives et y le label associé. Le Dataset d'entraînement est donc de la forme :

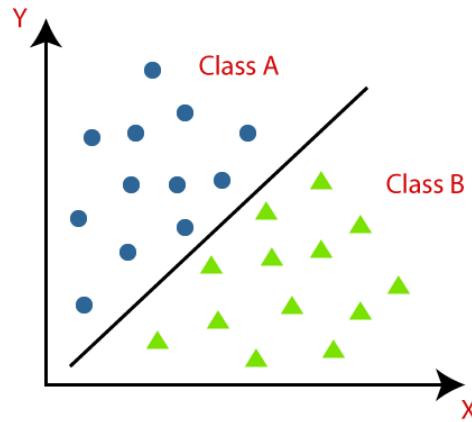
$$\mathcal{D} = \{(x_1, y_1), \dots, (x_n, y_n)\} \subseteq \mathcal{R}^d \times \mathcal{Y}$$

Objectif : Apprendre une fonction h t.q $y \approx h(x)$

Scénario	Espace $\mathcal{Y}$	Exemple (Finance)
Classification binaire	$\mathcal{Y} = \{0, 1\}$	Prédire si une action sera bullish (1) ou bearish (0).
Classification multiclasse	$\mathcal{Y} = \{C_1, C_2, \dots, C_K\}$	Catégoriser les entreprises selon leur niveau de risque (faible, moyen, haut).
Régression	$\mathcal{Y} \subseteq \mathbb{R}$	Prédire le rendement futur d'une action

Problèmes de Classification

Objectif : Apprendre $h : \mathcal{X} \rightarrow \mathcal{Y}$ avec $\mathcal{Y} \in \{0, 1\}$ (binaire) ou $\mathcal{Y} \in \{0, \dots, K\}$ (multiclass).



Principaux algorithmes : Régression logistique, Random Forest, k-NN, SVM, Gradient boosting...

Métriques d'évaluation : Précision, matrice de confusion, Recall, F-1 Score, AUROC...

Binary Cross Entropy (BCE) Loss :

- Soient $y_i \in \{0, 1\}$ la classe cible et $\hat{y}_i \in \{0, 1\}$ la probabilité prédite pour $y_i = 1$:

$$BCE_{cost_function} = -\frac{1}{N} \sum_{i=1}^N \overbrace{y_i \log(\hat{y}_i) + (1 - y_i) \log(1 - \hat{y}_i)}^{BCE_{Loss}}$$

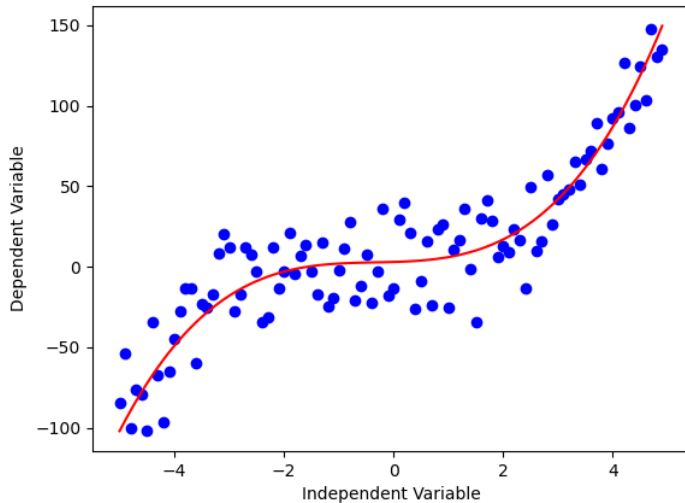
Categorical Cross-Entropy (CCE) Loss :

- Soient $y_i \in \{1, \dots, K\}$ la classe cible et $\hat{y}_i = [\hat{y}_{i1}, \hat{y}_{i2}, \dots, \hat{y}_{iK}]$ les probabilités prédites pour chaque classe :

$$CCE_{cost_function} = -\frac{1}{N} \sum_{i=1}^N \underbrace{\sum_{c=1}^K y_{i,c} \log(\hat{y}_{i,c})}_{CCE_{loss}}$$

Problèmes de Régression

Objectif : Apprendre $h : \mathcal{X} \rightarrow \mathcal{Y}$ avec $\mathcal{Y} \in \mathbb{R}$.



Principaux algorithmes : Régression linéaire, Ridge, Lasso, Random Forest, Gradient boosting, K-NN...

Métriques d'évaluation : MSE, R^2 , RMSE ...

Mean Square error (MSE):

- Soit $\hat{y}_i \in \mathbb{R}$ la valeur prédite et $y_i \in \mathbb{R}$ la valeur réelle :
$$MSE = \frac{1}{N} \sum_{i=1}^N \underbrace{(\hat{y}_i - y_i)^2}_{MSELoss}$$

Plan du cours

I. Introduction à l'apprentissage supervisé

- Problèmes de Classification
- Problèmes de Régression

II. Principaux modèles de Machine Learning

- Descente de Gradient
- Régressions linéaire et logistique
- Méthodes avancées

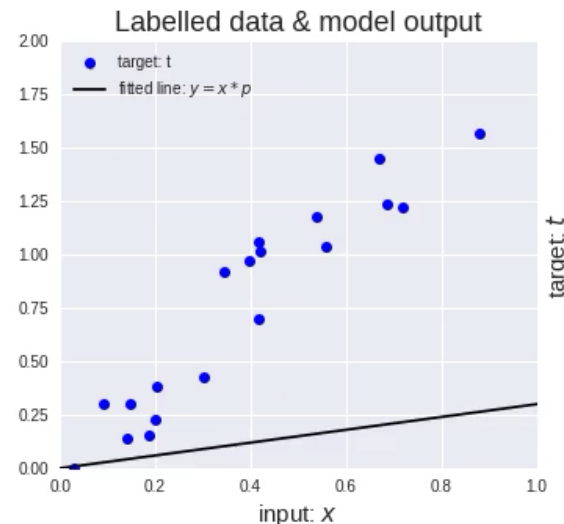
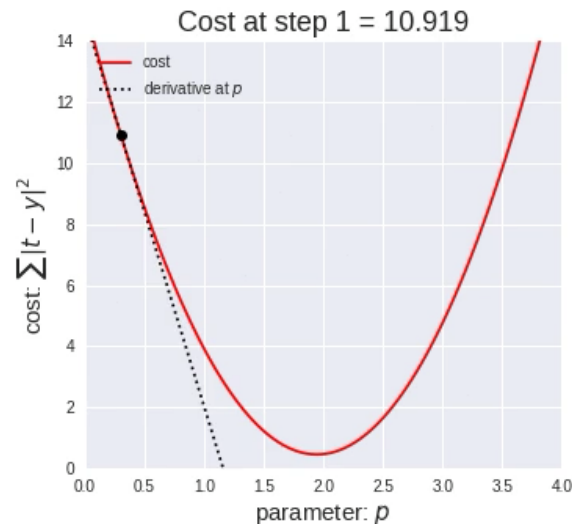
III. Métriques d'évaluation

IV. Explicabilité

Descente de Gradient

Algorithme de descente de gradient :

1. **Initialisation** des poids (aléatoire ou zéro)
2. Répéter jusqu'à convergence :
 1. **Prédiction** du modèle : $\hat{y} = \sigma(\theta x)$
 2. Calcul de l'**erreur** $L(\theta) = \frac{1}{n} \sum_{i=1}^n \ell(y_i, \hat{y}_i)$
 3. Calcul du **gradient de la fonction de coût** $\nabla L(\theta) = \frac{\partial L}{\partial \theta}$
 4. **Mise à jour des poids** (direction opposée au gradient) $\theta \leftarrow \theta - \alpha \nabla L(\theta)$
 5. **Critère d'arrêt** : $\max \text{itérations} |\nabla L(\theta)| \approx 0, \dots$



Méthodes de Régression

Objectif : Modéliser les relations entre les variables explicatives (x_j) et la variable cible y .

Formule :

- On veut déterminer la matrice de poids W telle que :

$$f(x) = X^T W \approx y \text{ avec } X = \begin{bmatrix} X \\ 1 \end{bmatrix} \text{ et } W = \begin{bmatrix} W' \\ b \end{bmatrix}$$

- **Estimation de W :** Formule explicite / Descente de Gradient.
- $f(x)$ est ensuite transformé (ou non) par une fonction d'activation en fonction du problème (Régression, Classification binaire/multiclasse).

Méthodes de Régression : Scénarios

Régression linéaire :

- **Prédiction** : $\hat{y} = f(x) = X^T W$
- **Fonction de coût** : *Mean Square Error (MSE)*
- **Optimisation des paramètres** : $w^* = (X^T X)^{-1} X^T y$

Régression logistique : Cas binaire

- **Prédiction** : $P(Y = 1|X = x) = \sigma(f(x)) = \frac{1}{1 + \exp(-(X^T W))}$
- **Fonction de coût** : *Binary Cross Entropy (BCE)*

Régression logistique : Cas multiclasse

- **Prédiction** : $P(Y = k|X = x) = \frac{\exp((X^T W_k))}{\sum_{j=1}^K \exp(x^T w_j)}$
- **Fonction de coût** : *Categorical Cross Entropy (CCE)*

Recommandations pratiques

Important pour une performance optimale :

- Pré-traiter les valeurs aberrantes (outliers).
- Normaliser les features (Standard Scaler).
- Eviter les trop fortes corrélations entre variables (méthodes de Régularisation).

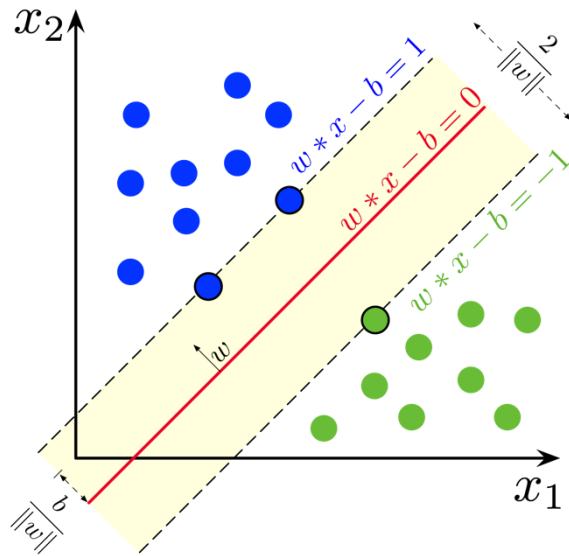
Inconvénients :

- Nécessite du feature engineering.
- Moins généralisable sur données complexes (haute dimension ou relations non linéaires).

Avantages :

- Facile à interpréter (les coefficients marquent l'impact des variables).
- Efficace sur petits datasets.

Support Vector Machine (SVM)



Objectif : Trouver l'hyperplan (d'équation $w^T x + b = 0$) qui sépare au mieux les classes en maximisant la marge $\frac{2}{\|w\|}$ entre les points les plus proches de différentes classes (*support vectors*).

Formule d'optimisations :

$$\min_{w,b} \frac{1}{2} \|w\|^2 \quad (\implies \text{Maximise la marge})$$

sous la contrainte : $y_i(w^T x_i + b) \geq 1$. $(\implies \text{Classification correcte})$

SVM : Kernel Trick

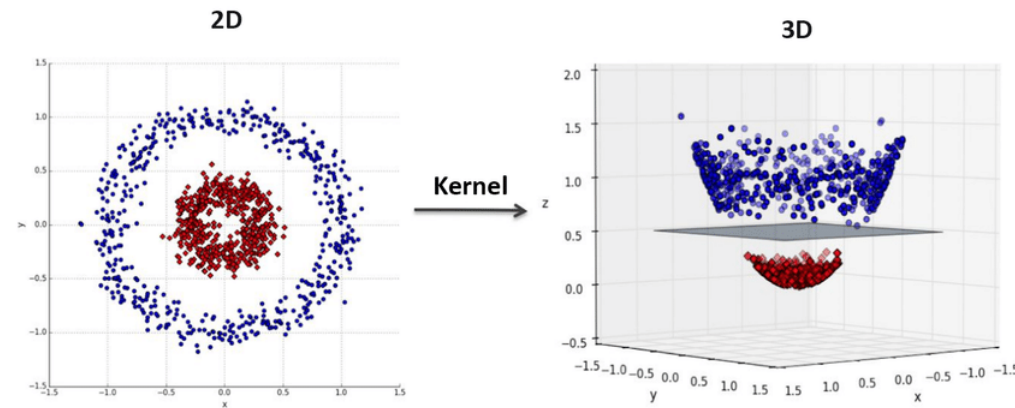
Si les données ne sont pas linéairement séparables :

Kernel Trick :

- Projeter les données dans un espace de dimension supérieure (dans lequel les données deviennent séparables).

➤ Principaux kernels :

- **Polynomial** : $K(x, y) = (x^T y + c)^d$
- **Gaussien** : $K(x, y) = \exp\left(\frac{-\|x - x'\|^2}{2\sigma}\right)$

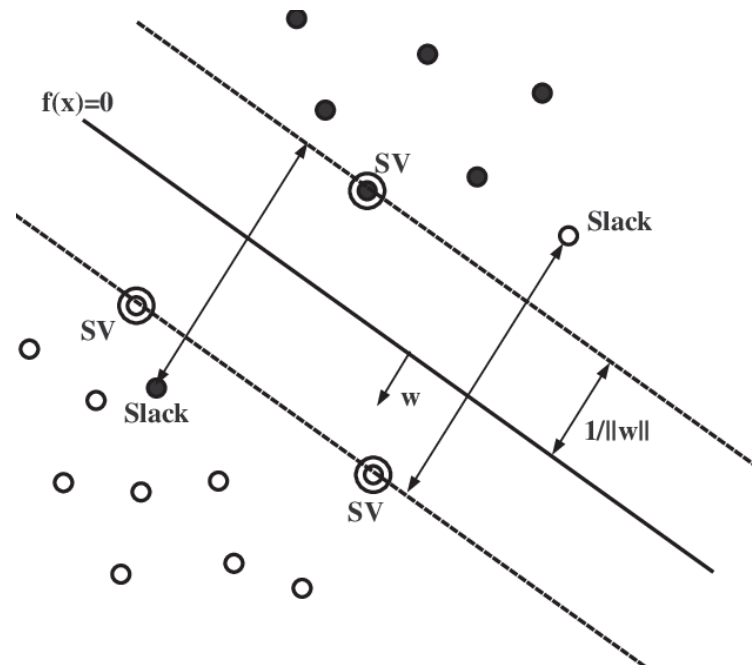


SVM : Marge souple

Marge souple : Permet une tolérance sur les mauvaises classification grâce à l'ajout de « slack variables ».

➤ **Nouvelle formule d'optimisation :**

$$\min_{w,b} \frac{1}{2} \|w\|^2 + C \sum_{i=1}^N \xi_i \quad \text{sous la contrainte : } y_i(w^T x_i + b) \geq 1 - \xi_i$$



SVM

Avantages :

- Robuste aux outliers.
- Efficace sur des données complexes et problèmes non linéaires.
- Fonctionne bien même si moins d'observations que de features.

Limites :

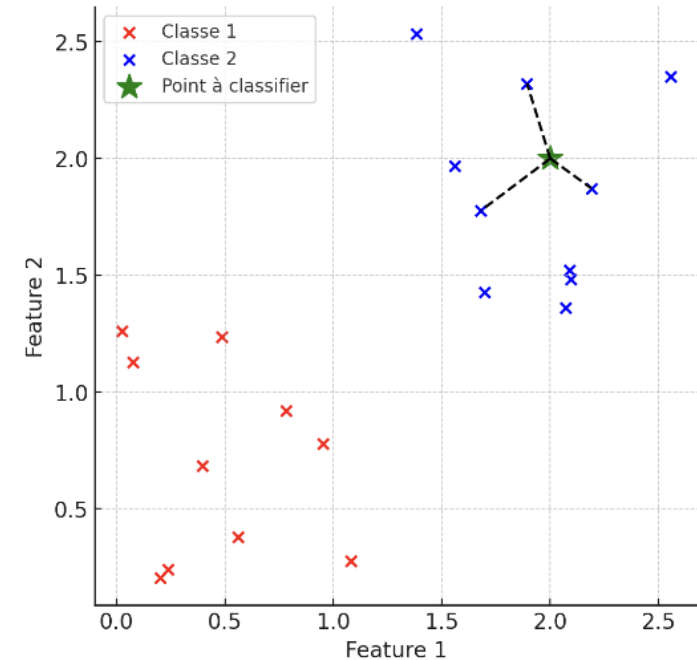
- Lent sur larges datasets (grande complexité)
- Sensible aux choix des hyperparamètres (*kernel*, *slack variables*...)
- Moins simple à interpréter que régression logistique.

K-Nearest Neighbours (KNN)

Intuition : Si un point x est très proche d'un point x' , il est probable qu'ils aient des valeurs cibles similaires.

Fonctionnement :

1. Initialiser K (nombre de voisins à considérer).
2. Calculer les distances (euclidienne, cosinus, mahalanobis...) entre le point à classifier et l'ensemble des points d'entraînements.
3. Sélectionner les K plus proches voisins.
4. Prédiction :
 - **Classification** : Classe majoritaire parmi les K plus proches voisins.
 - **Régression** : Moyenne des valeurs des K plus proches voisins.



KNN : Recommandations pratiques

Important pour une performance optimale :

- Normalisation des données (évite que certaines variables dominant trop fortement).
- Choix optimal de la distance en fonction des données (ex : Cosine si on s'intéresse juste aux angles des vecteurs...)
- Choix optimal de K (Validation croisée...)

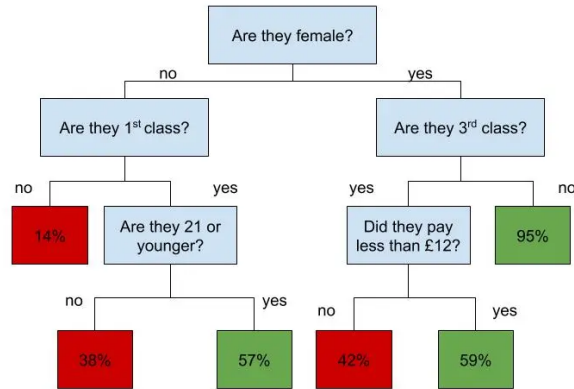
Avantages :

- Algorithme très simple à implémenter.
- Résultats facilement interprétables.

Inconvénients :

- Peu performant si très grand nombre de features.
- Complexité de calculs sur de grands datasets.

Arbre de décision



Principe : Diviser itérativement les données en fonction de différentes features afin de maximiser l'homogénéité de chaque sous-groupe à chaque étape.

Fonctionnement :

- **Racine :** Nœud contenant toutes les données.
- **Itérations :** Pour chaque nœud, on choisit la feature et la division qui réduit le plus l'impureté (hétérogénéité au sein d'un nœud) globale.
 - La division crée deux nœuds fils.
- **Fin :** Quand le critère de fin est atteint chaque feuille donne la classe (classification) ou la valeur (régression).

Arbre de décision : Hyperparamètres

Critères de décisions :

- **Classification :**

- **Critère de Gini :** $H(S) = 1 - \sum_i p_i^2$
- **Entropie :** $H(S) = - \sum_i p_i \log(p_i)$

- **Régression :**

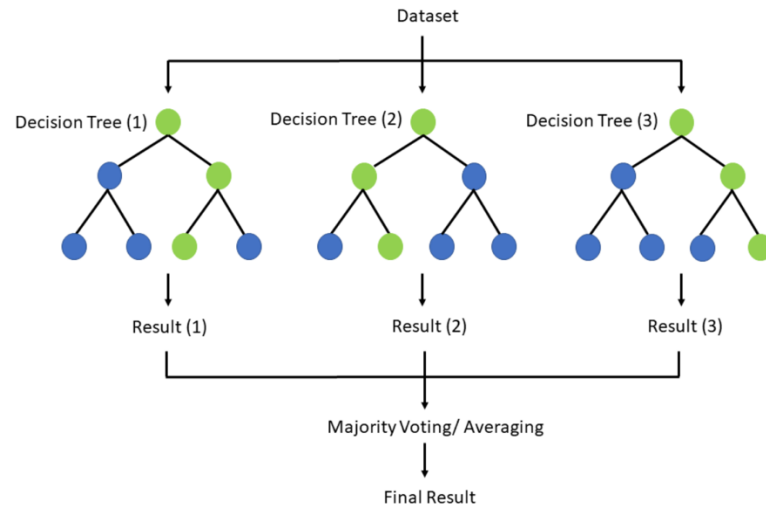
- **Variance :** $H(S) = \frac{1}{N} \sum_i (y_i - \bar{y})^2$

Gain d'information : $\Delta H = H_{parent} - \left(\frac{N_{gauche}}{N} H_{gauche} + \frac{N_{droit}}{N} H_{droit} \right)$

Critères d'arrêts (contrôlent l'overfitting):

- Profondeur max.
- Gain d'impureté min.
- Nombre de feuilles max.

Random Forest



Principe : Algorithme d'ensemble learning.

- On combine plusieurs arbres de décisions pour améliorer les performances et réduire l'overfitting.

Fonctionnement :

1. **Bootstrap sampling** : Génération de plusieurs sous échantillons aléatoires au sein du training set.
2. **Bagging** : Chaque arbre est entraîné sur un de ces échantillons aléatoires (tirage avec remise).
 - **Spécificité** : A chaque nœud, l'arbre ne teste qu'un sous ensemble des features.
3. **Prédiction** : Chaque arbre génère une prédiction. Aggrégation pour obtenir le résultat final.

Random Forest

Hyperparamètres : Nombre d'arbres et profondeur max des arbres.

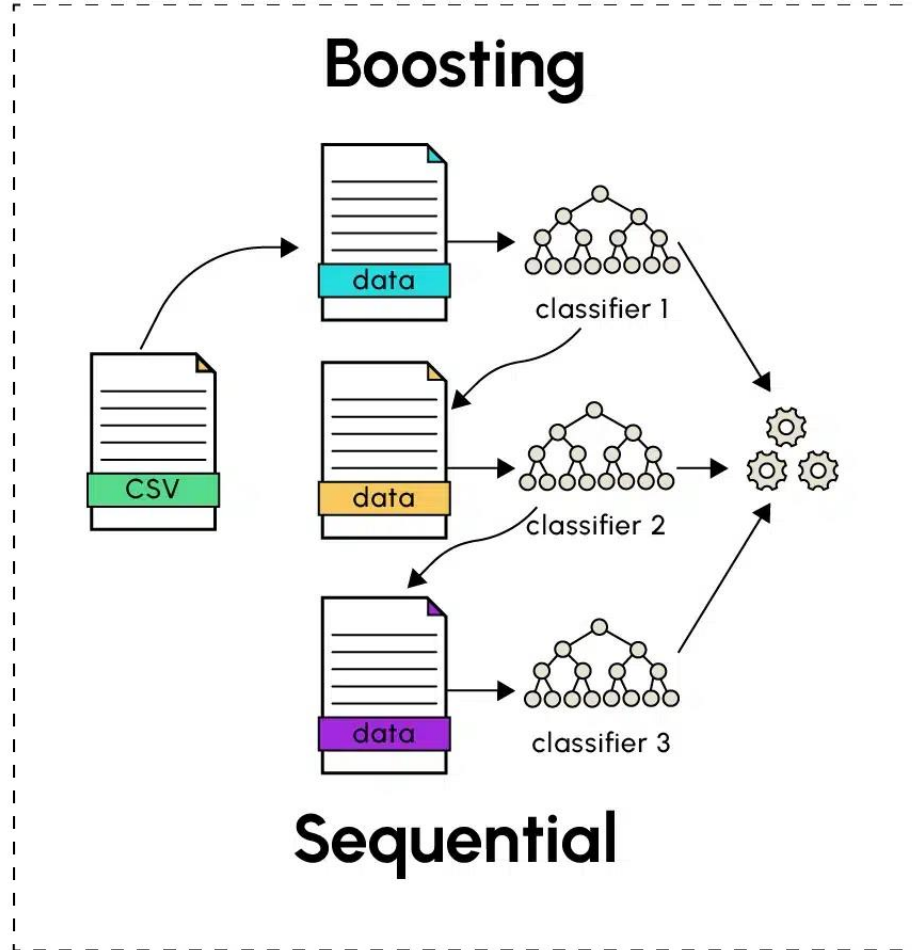
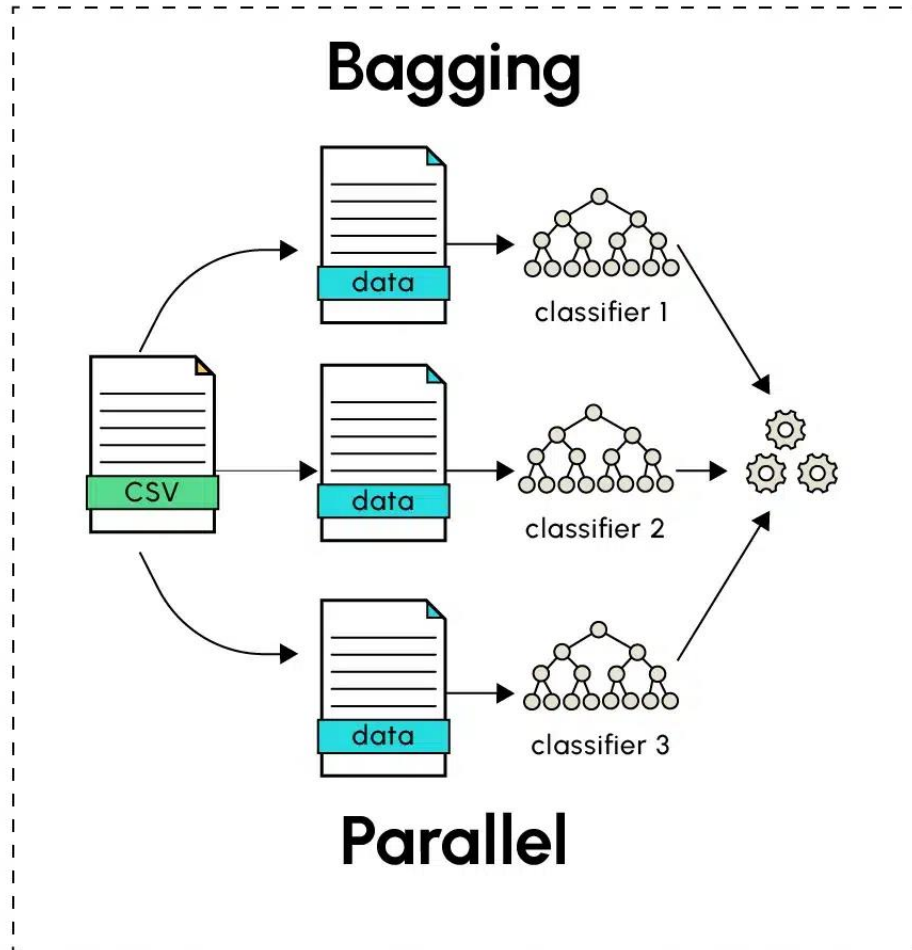
Avantages :

- Diversité permet de réduire l'overfitting.
- **Robustesse** : Aggrégation des résultats permet de réduire la variance.
- **Scalable** : Les arbres sont indépendants $\Rightarrow$ Entraînements parallèles $\Rightarrow$ Intéressant pour larges datasets.

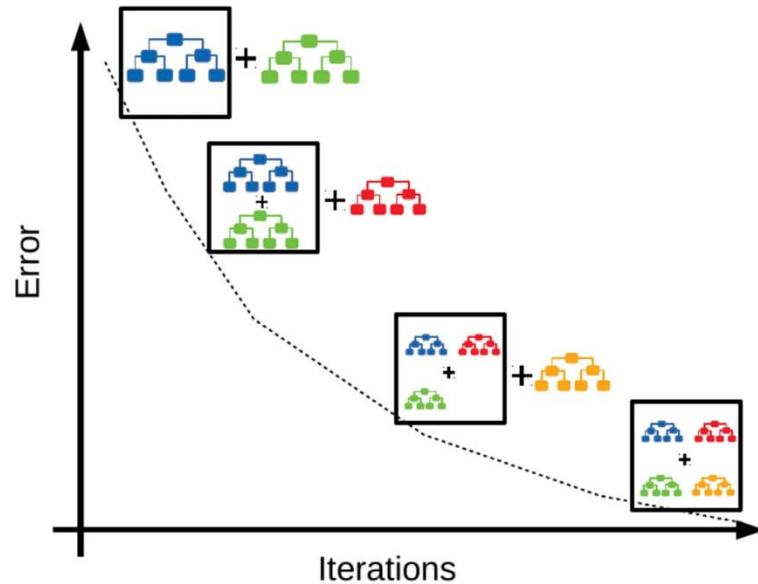
Inconvénients :

- Peu interprétable.
- Sensible aux déséquilibres dans les données (ex : Classe sur représentée)
 - Peut être traité avec SMOTE....

Bagging → Boosting



Gradient Boosting

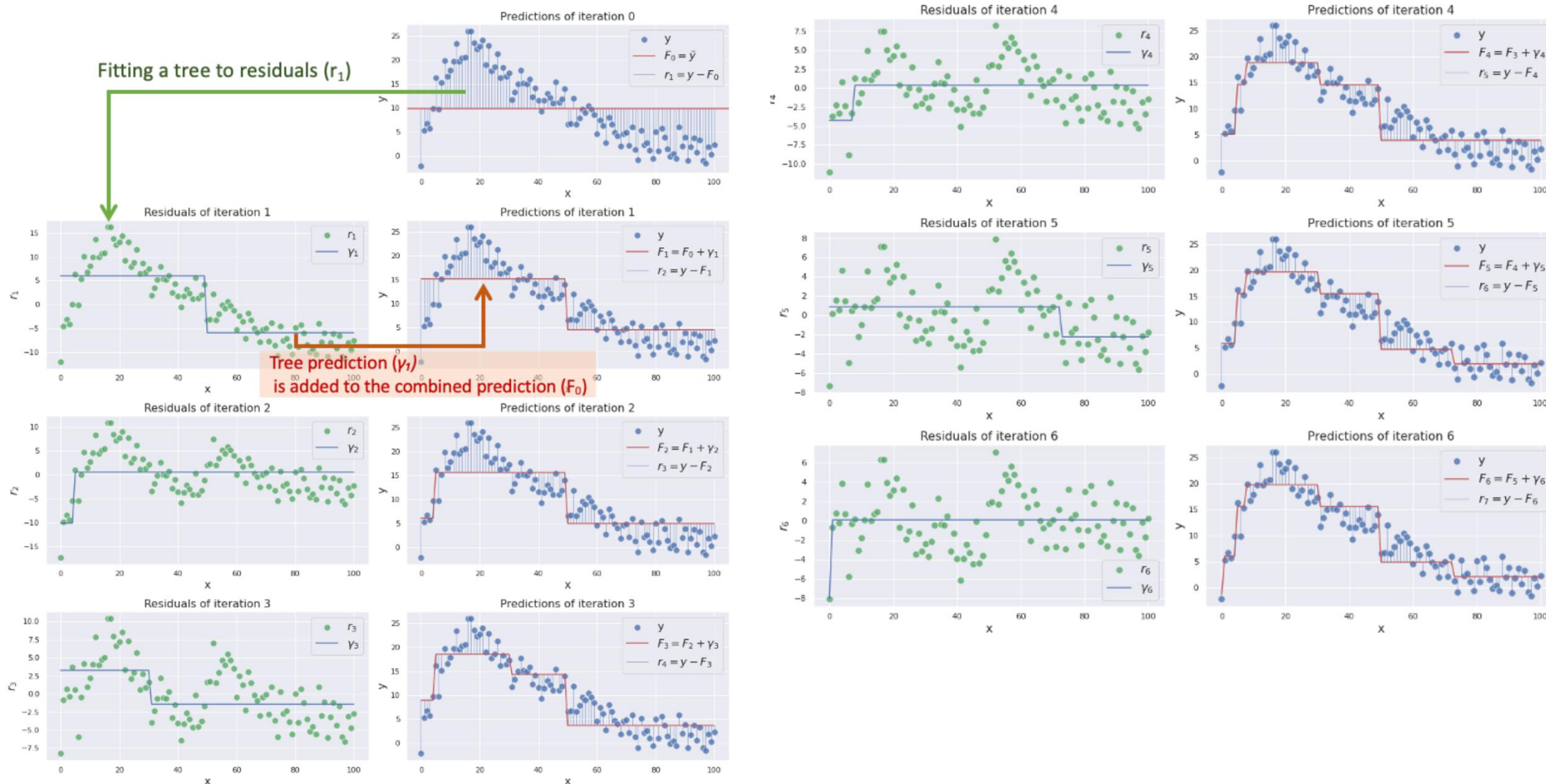


Principe : Chaque nouvel arbre est entraîné à corriger les erreurs du modèle précédent. Les prédictions de chaque arbre sont agrégées progressivement pour obtenir une prédiction précise.

Entraînement du modèle :

1. Premier modèle très simple (ex : Moyenne des valeurs cibles).
2. Calcul des erreurs du premier modèle.
3. Un nouvel arbre est entraîné à prédire ces erreurs.
4. Mise à jour des prédictions $F_{m+1}(x) = F_m(x) + \alpha h_m(x)$ avec $F_m(x)$ la prédiction des m premiers modèles et $h_m(x)$ la prédiction du modèle $m+1$ sur les erreurs.

Exemple : Régression



Gradient Boosting

Avantages :

- Algorithme le plus efficace (généralement) pour régression et classification.
- Peu sensible aux outliers.

Inconvénients :

- Peu interprétable.
- Entraînement plus lent et risque d'overfitting (atténué par **XGBoost**).

XGBoost : Algorithme de Gradient Boosting le plus utilisé.

- **Contrôle l'overfitting** : Critère d'arrêt, régularisation sur les poids des feuilles.
- **Accélère l'entraînement** : Parallélisation dans le choix des features.

Plan du cours

I. Introduction à l'apprentissage supervisé

- Problèmes de Classification
- Problèmes de Régression

II. Principaux modèles de Machine Learning

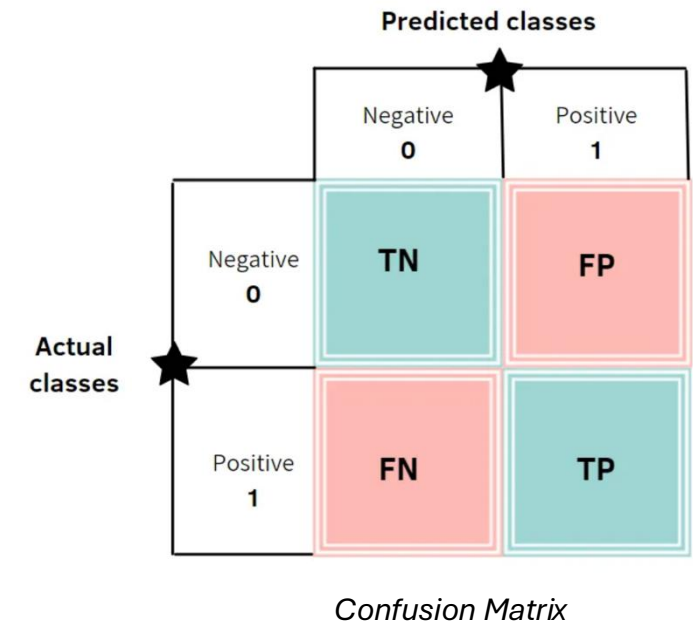
- Descente de Gradient
- Régressions linéaire et logistique
- Méthodes avancées

III. Métriques d'évaluation

IV. Explicabilité

Métriques d'évaluations : Classification

- **Accuracy** : $Accuracy = \frac{TP+TN}{TP+FP+FN+TN}$.
- **Precision** : $Precision = \frac{TP}{TP+FP} \Rightarrow$ Pertinent si un faux positif est problématique (ex : Spam).
- **Recall** : $Recall = \frac{TP}{TP+FN} \Rightarrow$ Pertinent si un faux négatif est problématique (ex : Spam).
- **F-1 Score** : $F1 = 2 \times \frac{Precision \times Recall}{Precision+Recall} \Rightarrow$ Pour prendre en compte les deux métriques.



Métriques d'évaluations : Régression

- **Mean Square Error (MSE)** : $MSE = \frac{1}{N} \sum_i (y_i - \hat{y}_i)^2$
 - **Mean Absolute Error (MAE)** : $MAE = \frac{1}{N} \sum_i |y_i - \hat{y}_i|$
 - **RMSE** : $RMSE = \sqrt{\frac{1}{N} \sum_i (y_i - \hat{y}_i)^2}$
- 
- Permettent d'interpréter l'erreur dans la même unité que les données cibles.
 - RMSE pénalise plus les grandes erreurs.

Plan du cours

I. Introduction à l'apprentissage supervisé

- Problèmes de Classification
- Problèmes de Régression

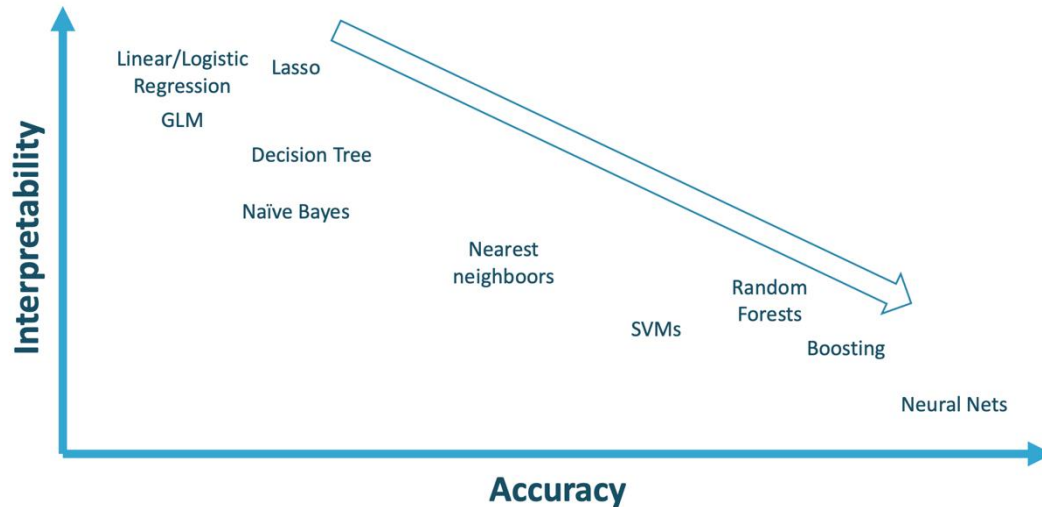
II. Principaux modèles de Machine Learning

- Descente de Gradient
- Régressions linéaire et logistique
- Méthodes avancées

III. Métriques d'évaluation

IV. Explicabilité

Explicabilité



Problématique : Les modèles les plus performants (xgboost...) sont des **boîtes noires**, on ne peut pas interpréter leurs résultats.

Enjeux de l'explicabilité des modèles :

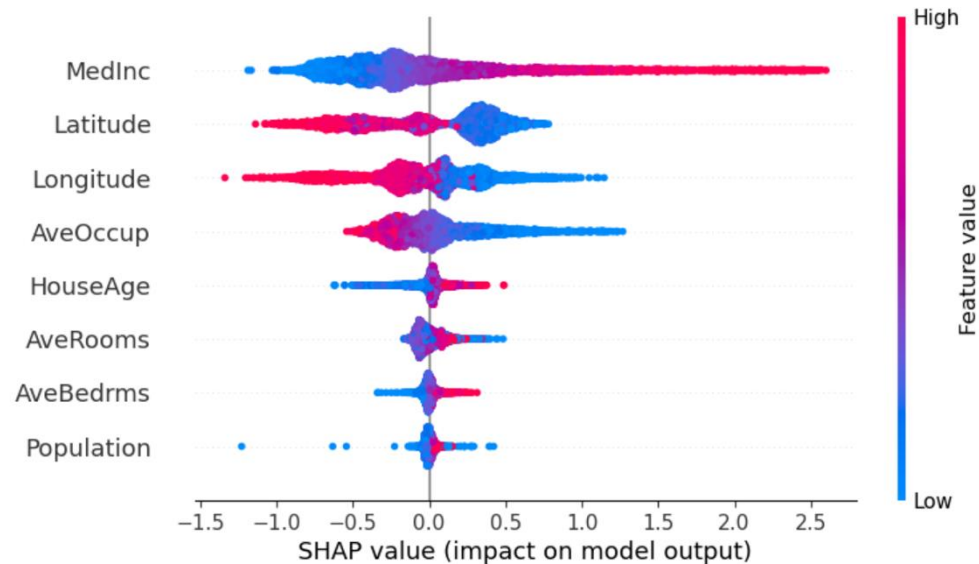
- **Conformité réglementaire :** Régulations sur l'usage de l'IA.
 - Explicabilité souvent requise pour le déploiement d'un modèle.
- **Transparence :** Un manque de transparence peut entraîner un manque de confiance sur les décisions.
- **Meilleure interprétation des résultats :** Permet d'identifier les variables les plus importantes et leurs impacts sur les décisions du modèle.

Shapley Additive Explanations (SHAP)

Objectif : Expliquer les prédictions des modèles « boîtes noires » en identifiant l'impact de chaque variable.

Valeur de Shapley :

- La valeur de shapley d'une variable correspond à sa contribution marginale moyenne si on prend tous les sous-ensembles possibles de variables.



Shap summary – Dataset : California House – **Variable cible :** Prix des logements.